

Lab 2: Building the Virtual FANUC

EE0849 Introduction to Robotics

Basri Erdogan

Table of contents

1	Overview	2
2	Learning Objectives	2
3	Pre-Lab: DH Convention Review	3
3.1	DH Parameter Definitions	3
3.2	The DH Transformation Matrix	3
3.3	Pre-Lab Questions	3
4	Part 1: 2-Link Planar Arm (30 min)	4
4.1	1.1 Problem Setup	4
4.2	1.2 DH Parameter Table	4
4.3	1.3 Building the Model	4
4.4	1.4 Forward Kinematics Verification	5
4.5	1.5 Exercise 1: Verify Your Pre-Lab Calculation	5
4.6	1.6 Visualization	6
5	Part 2: Understanding the FANUC LR Mate 200iD (20 min)	6
5.1	2.1 FANUC LR Mate 200iD Specifications	6
5.2	2.2 Robot Geometry	7
5.3	2.3 Joint Configuration	7
5.4	2.4 Exercise 2: Sketch the Robot	7
6	Part 3: Building the FANUC Model (40 min)	7
6.1	3.1 DH Parameter Table for FANUC LR Mate 200iD	7
6.2	3.2 Building the Model in Python	8
6.3	3.3 Verify Zero Configuration	9
6.4	3.4 Exercise 3: Calculate Expected Zero Position	9
6.5	3.5 Visualization	9
6.6	3.6 Test Different Configurations	10
7	Part 4: Workspace Visualization (20 min)	10
7.1	4.1 What is Workspace?	10

7.2	4.2 Generating Workspace Points	10
7.3	4.3 Visualizing the Workspace	11
7.4	4.4 Exercise 4: Workspace Slice	12
8	Part 5: Interactive Exploration (10 min)	13
8.1	5.1 Using teach() for Interactive Control	13
8.2	5.2 Exercise 5: Find Specific Poses	13
9	Deliverables	14
9.1	Required Files	14
9.2	Submission Format	14
10	Grading Rubric	14
11	Challenge Problems (Optional)	15
11.1	Challenge 1: Add Joint Limits	15
11.2	Challenge 2: Compare with Built-in Model	15
11.3	Challenge 3: Workspace Volume	15
12	Additional Resources	16
13	Appendix: Common Issues	16
13.1	Robot plot looks wrong	16
13.2	Units mismatch	16
13.3	teach() doesn't open	16

1 Overview

In this lab, you will apply the Denavit-Hartenberg (DH) convention to build virtual robot models in Python. You'll start with simple planar arms to verify your understanding, then progress to modeling the FANUC LR Mate 200iD industrial robot using its official datasheet dimensions.

Duration: 2 hours

Software Required:

- Python environment from Lab 0
- Libraries: `roboticstoolbox-python`, `spatialmath-python`, `numpy`, `matplotlib`

2 Learning Objectives

By the end of this lab, you will be able to:

1. Extract DH parameters from robot geometry
2. Build `DHRobot` models using `roboticstoolbox-python`

3. Compute forward kinematics using `fkine()`
4. Verify robot models against known configurations
5. Visualize robot workspace and reachability

3 Pre-Lab: DH Convention Review

Complete this section **before** coming to lab.

3.1 DH Parameter Definitions

Recall the four DH parameters (Standard DH convention):

Parameter	Symbol	Description
Link length	a_i	Distance along x_i from z_{i-1} to z_i
Link twist	α_i	Angle about x_i from z_{i-1} to z_i
Link offset	d_i	Distance along z_{i-1} from x_{i-1} to x_i
Joint angle	θ_i	Angle about z_{i-1} from x_{i-1} to x_i

For a **revolute** joint: θ_i is the variable, d_i is constant.

For a **prismatic** joint: d_i is the variable, θ_i is constant.

3.2 The DH Transformation Matrix

The transformation from frame $i-1$ to frame i is:

$${}^{i-1}T_i = \begin{bmatrix} c\theta_i & -s\theta_i c\alpha_i & s\theta_i s\alpha_i & a_i c\theta_i \\ s\theta_i & c\theta_i c\alpha_i & -c\theta_i s\alpha_i & a_i s\theta_i \\ 0 & s\alpha_i & c\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where $c = \cos$, $s = \sin$.

3.3 Pre-Lab Questions

Answer these before lab (write in your lab notebook):

1. For a 2-link planar arm with link lengths $L_1 = 1$ m and $L_2 = 0.5$ m, write the DH parameter table.
2. What does $\alpha = 90^\circ$ mean physically?

3. If joint 1 = 30° and joint 2 = 45° for your 2-link arm, calculate the end-effector position by hand.
-

4 Part 1: 2-Link Planar Arm (30 min)

Let's start with a simple robot to verify our understanding.

4.1 1.1 Problem Setup

Consider a 2-link planar arm in the XY plane:

- Link 1: length $L_1 = 1.0$ m
- Link 2: length $L_2 = 0.5$ m
- Both joints are revolute
- Robot operates in the XY plane (all $\alpha_i = 0$, all $d_i = 0$)

4.2 1.2 DH Parameter Table

For a planar arm, the DH table is straightforward:

Joint	θ_i	d_i	a_i	α_i
1	θ_1	0	1.0	0
2	θ_2	0	0.5	0

4.3 1.3 Building the Model

```
import numpy as np
from roboticstoolbox import DHRobot, RevoluteDH
import matplotlib.pyplot as plt

# Define DH parameters for 2-link planar arm
L1 = 1.0 # meters
L2 = 0.5 # meters

# Create links using RevoluteDH(d, a, alpha)
# Note: theta is the variable for revolute joints
link1 = RevoluteDH(d=0, a=L1, alpha=0)
link2 = RevoluteDH(d=0, a=L2, alpha=0)

# Create the robot
planar_2link = DHRobot([link1, link2], name='Planar-2R')

# Print robot info
```

```
print(planar_2link)
```

4.4 1.4 Forward Kinematics Verification

```
# Test case 1: Zero configuration (all joints = 0)
q_zero = [0, 0]
T_zero = planar_2link.fkine(q_zero)
print("Zero configuration:")
print(f"  Joint angles: {q_zero}")
print(f"  End-effector position: {T_zero.t}")
print(f"  Expected: [1.5, 0, 0]")

# Test case 2: q1 = 90°, q2 = 0°
q_test1 = [np.pi/2, 0]
T_test1 = planar_2link.fkine(q_test1)
print("\nq1=90°, q2=0°:")
print(f"  End-effector position: {T_test1.t}")
print(f"  Expected: [0, 1.5, 0]")

# Test case 3: q1 = 0°, q2 = 90°
q_test2 = [0, np.pi/2]
T_test2 = planar_2link.fkine(q_test2)
print("\nq1=0°, q2=90°:")
print(f"  End-effector position: {T_test2.t}")
print(f"  Expected: [1.0, 0.5, 0]")
```

4.5 1.5 Exercise 1: Verify Your Pre-Lab Calculation

Task: Verify your pre-lab calculation ($q_1 = 30^\circ$, $q_2 = 45^\circ$) using Python.

```
# Your code here
q_prelab = [np.radians(30), np.radians(45)]
T_prelab = planar_2link.fkine(q_prelab)

print("Pre-lab verification:")
print(f"  Joint angles: q1={30}°, q2={45}°")
print(f"  Computed position: {T_prelab.t}")

# Hand calculation expected values:
# x = L1*cos(30) + L2*cos(30+45) = 1*0.866 + 0.5*cos(75°) = 0.866 + 0.129 = 0.995
# y = L1*sin(30) + L2*sin(30+45) = 1*0.5 + 0.5*sin(75°) = 0.5 + 0.483 = 0.983
print(f"  Hand calculation: [{1*np.cos(np.radians(30)) + 0.5*np.cos(np.radians(75)):.3f}, "
      f"{1*np.sin(np.radians(30)) + 0.5*np.sin(np.radians(75)):.3f}, 0]")
```

Record: Do your hand calculations match the Python result?

4.6 1.6 Visualization

```
# Plot the robot in different configurations
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

configurations = [
    ([0, 0], "Zero Config"),
    ([np.pi/2, 0], "q1=90°"),
    ([np.pi/4, np.pi/4], "q1=q2=45°")
]

for ax, (q, title) in zip(axes, configurations):
    planar_2link.plot(q, ax=ax, block=False)
    ax.set_title(title)
    ax.set_xlim([-2, 2])
    ax.set_ylim([-2, 2])

plt.tight_layout()
plt.savefig('planar_2link_configs.png', dpi=150)
plt.show()
```

5 Part 2: Understanding the FANUC LR Mate 200iD (20 min)

Now let's examine a real industrial robot.

5.1 2.1 FANUC LR Mate 200iD Specifications

The FANUC LR Mate 200iD is a 6-DOF industrial robot commonly used for:

- Assembly operations
- Material handling
- Machine tending
- Small parts picking

Key Specifications:

Parameter	Value
Degrees of Freedom	6 (all revolute)
Payload	7 kg
Reach	717 mm
Repeatability	±0.02 mm

5.2 2.2 Robot Geometry

The FANUC LR Mate 200iD has the following key dimensions (from datasheet):

Dimension	Value (mm)	Description
d_1	330	Base height
a_2	50	Link 2 offset
a_3	330	Upper arm length
a_4	35	Forearm offset
d_4	335	Forearm length
d_6	80	End-effector offset

5.3 2.3 Joint Configuration

The robot has a typical anthropomorphic structure:

- **Joint 1 (Waist):** Rotation about vertical axis
- **Joint 2 (Shoulder):** Rotation of upper arm
- **Joint 3 (Elbow):** Rotation of forearm
- **Joints 4, 5, 6 (Wrist):** Spherical wrist for orientation

5.4 2.4 Exercise 2: Sketch the Robot

Task: On paper, sketch the FANUC robot showing:

1. The base frame $\{0\}$
2. Each joint axis
3. Approximate locations of frames $\{1\}$ through $\{6\}$

Use the dimensions table above. This will help you understand the DH parameters.

6 Part 3: Building the FANUC Model (40 min)

6.1 3.1 DH Parameter Table for FANUC LR Mate 200iD

Based on the robot geometry and following the DH convention:

Joint	θ_i	d_i (mm)	a_i (mm)	α_i (deg)
1	θ_1	330	50	-90
2	θ_2	0	330	0
3	θ_3	0	35	-90
4	θ_4	335	0	90
5	θ_5	0	0	-90
6	θ_6	80	0	0

Joint	θ_i	d_i (mm)	a_i (mm)	α_i (deg)
-------	------------	------------	------------	------------------

i Units Matter!

The toolbox expects consistent units. We'll use **meters** for all lengths.

6.2 3.2 Building the Model in Python

```
import numpy as np
from roboticstoolbox import DHRobot, RevoluteDH
from spatialmath import SE3

# FANUC LR Mate 200iD dimensions (converted to meters)
d1 = 0.330 # Base height
a1 = 0.050 # Link 1 offset
a2 = 0.330 # Upper arm length
a3 = 0.035 # Elbow offset
d4 = 0.335 # Forearm length
d6 = 0.080 # End-effector offset

# Create DH links
# RevoluteDH(d, a, alpha, offset=0)
# offset is added to theta for home position adjustment

links = [
    RevoluteDH(d=d1, a=a1, alpha=-np.pi/2), # Joint 1
    RevoluteDH(d=0, a=a2, alpha=0),         # Joint 2
    RevoluteDH(d=0, a=a3, alpha=-np.pi/2),  # Joint 3
    RevoluteDH(d=d4, a=0, alpha=np.pi/2),   # Joint 4
    RevoluteDH(d=0, a=0, alpha=-np.pi/2),   # Joint 5
    RevoluteDH(d=d6, a=0, alpha=0),         # Joint 6
]

# Create the robot
fanuc = DHRobot(links, name='FANUC LR Mate 200iD')

# Print robot information
print(fanuc)
print("\nDH Parameters:")
print(fanuc.dyntable())
```

6.3 3.3 Verify Zero Configuration

```
# Zero configuration: all joints at 0
q_zero = [0, 0, 0, 0, 0, 0]

T_zero = fanuc.fkine(q_zero)

print("Zero Configuration Analysis:")
print(f"  Joint angles: {np.degrees(q_zero)} degrees")
print(f"\nEnd-effector pose:")
print(T_zero)
print(f"\nPosition (x, y, z): {T_zero.t} meters")
print(f"Position (mm): {T_zero.t * 1000}")
```

6.4 3.4 Exercise 3: Calculate Expected Zero Position

Task: By hand, calculate what the end-effector position should be in the zero configuration.

Hint: Sum up the contributions from each link:

- d_1 adds height in Z
- With $\alpha = -90^\circ$ at joint 1, subsequent links extend horizontally
- Trace through each transformation

```
# Expected position at zero config (approximately):
# The robot should be "stretched out" horizontally
# Z = d1 = 330 mm (base height)
# X = a1 + a2 + a3 + d4 + d6 = 50 + 330 + 35 + 335 + 80 = 830 mm

print("\nExpected (approximate):")
print(f"  X   {50 + 330 + 35 + 335 + 80} mm")
print(f"  Z   {330} mm")
print("\nNote: Actual values depend on twist angles affecting direction")
```

6.5 3.5 Visualization

```
import matplotlib.pyplot as plt

# Plot zero configuration
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

fanuc.plot(q_zero, ax=ax, block=False)

ax.set_xlabel('X (m)')
```

```

ax.set_ylabel('Y (m)')
ax.set_zlabel('Z (m)')
ax.set_title('FANUC LR Mate 200iD - Zero Configuration')

plt.savefig('fanuc_zero_config.png', dpi=150)
plt.show()

```

6.6 3.6 Test Different Configurations

```

# Define several test configurations
test_configs = {
    'zero': [0, 0, 0, 0, 0, 0],
    'shoulder_up': [0, -np.pi/4, 0, 0, 0, 0],
    'elbow_bent': [0, 0, np.pi/2, 0, 0, 0],
    'waist_turn': [np.pi/2, 0, 0, 0, 0, 0],
    'wrist_pitch': [0, 0, 0, 0, np.pi/4, 0],
}

print("Configuration Analysis:")
print("=" * 60)

for name, q in test_configs.items():
    T = fanuc.fkine(q)
    pos = T.t * 1000 # Convert to mm
    print(f"\n{name}:")
    print(f"  Joints (deg): {np.round(np.degrees(q), 1)}")
    print(f"  Position (mm): [{pos[0]:.1f}, {pos[1]:.1f}, {pos[2]:.1f}]")

```

7 Part 4: Workspace Visualization (20 min)

7.1 4.1 What is Workspace?

The **workspace** of a robot is the set of all positions the end-effector can reach. For a 6-DOF robot:

- **Reachable workspace:** All positions that can be reached with at least one orientation
- **Dexterous workspace:** Positions reachable with any orientation

7.2 4.2 Generating Workspace Points

```

import numpy as np
from roboticstoolbox import DHRobot, RevoluteDH

```

```

# Use our FANUC model (recreate if needed)
# ... (model definition from above)

# Generate random configurations
np.random.seed(42) # For reproducibility
n_samples = 5000

# Joint limits for FANUC (approximate, in radians)
joint_limits = [
    (-2.97, 2.97), # Joint 1: ±170°
    (-1.05, 2.79), # Joint 2: -60° to +160°
    (-2.44, 4.19), # Joint 3: -140° to +240°
    (-3.32, 3.32), # Joint 4: ±190°
    (-2.09, 2.09), # Joint 5: ±120°
    (-6.28, 6.28), # Joint 6: ±360°
]

# Sample random configurations within joint limits
positions = []

for _ in range(n_samples):
    q = [np.random.uniform(low, high) for (low, high) in joint_limits]
    T = fanuc.fkine(q)
    positions.append(T.t)

positions = np.array(positions)

```

7.3 4.3 Visualizing the Workspace

```

import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(12, 5))

# 3D view
ax1 = fig.add_subplot(121, projection='3d')
ax1.scatter(positions[:, 0], positions[:, 1], positions[:, 2],
            c=positions[:, 2], cmap='viridis', alpha=0.3, s=1)
ax1.set_xlabel('X (m)')
ax1.set_ylabel('Y (m)')
ax1.set_zlabel('Z (m)')
ax1.set_title('FANUC Workspace (3D)')

# Top-down view (XY plane)

```

```

ax2 = fig.add_subplot(122)
ax2.scatter(positions[:, 0], positions[:, 1],
            c=positions[:, 2], cmap='viridis', alpha=0.3, s=1)
ax2.set_xlabel('X (m)')
ax2.set_ylabel('Y (m)')
ax2.set_title('Workspace (Top View)')
ax2.axis('equal')
ax2.colorbar = plt.colorbar(ax2.collections[0], ax=ax2, label='Z (m)')

plt.tight_layout()
plt.savefig('fanuc_workspace.png', dpi=150)
plt.show()

# Calculate workspace statistics
print("\nWorkspace Statistics:")
print(f" X range: [{positions[:, 0].min():.3f}, {positions[:, 0].max():.3f}] m")
print(f" Y range: [{positions[:, 1].min():.3f}, {positions[:, 1].max():.3f}] m")
print(f" Z range: [{positions[:, 2].min():.3f}, {positions[:, 2].max():.3f}] m")
print(f" Max reach: {np.linalg.norm(positions, axis=1).max():.3f} m")

```

7.4 4.4 Exercise 4: Workspace Slice

Task: Create a 2D plot showing the workspace in the XZ plane (side view) for $Y = 0$.

```

# Filter points near Y = 0 (within ±50mm)
y_tolerance = 0.05 # 50 mm
mask = np.abs(positions[:, 1]) < y_tolerance
slice_positions = positions[mask]

plt.figure(figsize=(8, 6))
plt.scatter(slice_positions[:, 0], slice_positions[:, 2], alpha=0.5, s=2)
plt.xlabel('X (m)')
plt.ylabel('Z (m)')
plt.title('Workspace Slice at Y = 0')
plt.axis('equal')
plt.grid(True)
plt.savefig('fanuc_workspace_slice.png', dpi=150)
plt.show()

print(f"\nPoints in slice: {len(slice_positions)} out of {len(positions)}")

```

8 Part 5: Interactive Exploration (10 min)

8.1 5.1 Using teach() for Interactive Control

The robotics toolbox provides an interactive teaching pendant:

```
# Interactive visualization (opens a new window)
# Note: This requires a GUI backend

fanuc.teach(q_zero)
```

i GUI Note

The `teach()` function opens an interactive window with sliders for each joint. If this doesn't work in your environment, use the plotting methods instead.

8.2 5.2 Exercise 5: Find Specific Poses

Task: Use trial and error (or the teach pendant) to find joint configurations that place the end-effector at approximately:

1. Position A: (0.5, 0.0, 0.4) meters
2. Position B: (0.3, 0.3, 0.5) meters

Record the joint angles you find. There may be multiple solutions (this is the inverse kinematics problem we'll study later!).

```
# Template for testing configurations
def test_config(q, target_name, target_pos):
    T = fanuc.fkine(q)
    pos = T.t
    error = np.linalg.norm(pos - np.array(target_pos))

    print(f"\n{target_name}:")
    print(f"  Target: {target_pos}")
    print(f"  Achieved: [{pos[0]:.3f}, {pos[1]:.3f}, {pos[2]:.3f}]")
    print(f"  Error: {error*1000:.1f} mm")
    print(f"  Joints (deg): {np.round(np.degrees(q), 1)}")

# Your solution for Position A
q_A = [0, 0, 0, 0, 0, 0] # Modify this!
test_config(q_A, "Position A", [0.5, 0.0, 0.4])

# Your solution for Position B
q_B = [0, 0, 0, 0, 0, 0] # Modify this!
test_config(q_B, "Position B", [0.3, 0.3, 0.5])
```

9 Deliverables

Submit the following by the deadline:

9.1 Required Files

1. **lab2_fanuc.py** - Python file containing:
 - Your 2-link planar arm model
 - Your FANUC LR Mate 200iD model
 - Solutions to all exercises
 - All verification code
2. **Figures (PNG format):**
 - **planar_2link_configs.png** - 2-link arm in 3 configurations
 - **fanuc_zero_config.png** - FANUC at zero configuration
 - **fanuc_workspace.png** - Workspace visualization
 - **fanuc_workspace_slice.png** - Workspace XZ slice
3. **lab2_report.pdf** - Short report (2-3 pages) containing:
 - Pre-lab calculations and sketch
 - DH parameter table for FANUC (handwritten or typed)
 - Screenshot of your robot model
 - Answers to: Does your hand calculation match Python?
 - Exercise 5: The joint configurations you found
 - Comparison of calculated vs. measured workspace reach

9.2 Submission Format

- Combine all files into a ZIP archive: **Lab2_YourName.zip**
 - Submit via course portal
-

10 Grading Rubric

Component	Points
Pre-lab questions and sketch	10
Exercise 1: 2-link verification	15
Exercise 2: Robot sketch	10
Exercise 3: Zero position calculation	15
Exercise 4: Workspace slice plot	15
Exercise 5: Finding configurations	15
FANUC model code quality	10
Report completeness	10
Total	100

Component	Points
-----------	--------

11 Challenge Problems (Optional)

For extra practice, try these advanced problems:

11.1 Challenge 1: Add Joint Limits

Modify your FANUC model to include proper joint limits:

```
links = [
    RevoluteDH(d=d1, a=a1, alpha=-np.pi/2, qlim=[-2.97, 2.97]),
    # ... add limits to all joints
]
```

Then verify that `fanuc.islimit(q)` correctly identifies out-of-range configurations.

11.2 Challenge 2: Compare with Built-in Model

The robotics toolbox has built-in models. Compare yours:

```
import roboticstoolbox as rtb

# Load built-in FANUC model
fanuc_builtin = rtb.models.DH.FANUC()

# Compare forward kinematics
q_test = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6]
T_yours = fanuc.fkine(q_test)
T_builtin = fanuc_builtin.fkine(q_test)

print("Position difference (mm):", np.linalg.norm(T_yours.t - T_builtin.t) * 1000)
```

11.3 Challenge 3: Workspace Volume

Estimate the volume of the robot's workspace using the Monte Carlo method:

```
# Count points in a bounding box vs. total samples
# Volume (points inside / total) × bounding box volume
```

12 Additional Resources

- **FANUC Robot Datasheet:** Available on course portal
 - **roboticstoolbox documentation:** petercorke.github.io/robotics-toolbox-python
 - **DH Convention Tutorial:** Spong, Hutchinson, Vidyasagar Chapter 3
-

13 Appendix: Common Issues

13.1 Robot plot looks wrong

Make sure your DH parameters are in the correct order:

```
RevoluteDH(d=..., a=..., alpha=...) # NOT (a, d, alpha)
```

13.2 Units mismatch

Always convert mm to meters before building the model:

```
d1 = 330 / 1000 # or d1 = 0.330
```

13.3 teach() doesn't open

Try using a different matplotlib backend:

```
import matplotlib
matplotlib.use('TkAgg') # or 'Qt5Agg'
```

Or use Swift backend for better 3D visualization:

```
fanuc.plot(q, backend='swift')
```